

13.1 How to read this chapter

The USB OTG controller is available in the LPC1768, LPC1766, LPC1765, LPC1758, LPC1756, and LPC1754. On these devices, the USB controller can be configured for device, Host, or OTG operation.

13.2 Basic configuration

The USB controller is configured using the following registers:

1. Power: In the PCONP register ([Table 46](#)), set bit PCUSB.
Remark: On reset, the USB block is disabled (PCUSB = 0).
2. Clock: The USB clock can be generated using the dedicated USB PLL (PLL1) or with the Main PLL (PLL0). See [Section 4.6.1](#).
3. Pins: Select USB pins and their modes in PINSEL0 to PINSEL5 and PINMODE0 to PINMODE5 ([Section 8.5](#)).
4. Wake-up: Activity on the USB bus port can wake up the microcontroller from Power-down mode (see [Section 13.10.2](#) and [Section 4.8.8](#)).
5. Interrupts: Interrupts are enabled in the NVIC using the appropriate Interrupt Set Enable register.
6. Initialization: see [Section 13.11](#).

13.3 Introduction

Remark: The LPC175x devices require a standard I²C connection to the USB ATX.

This chapter describes the OTG and I²C portions of the USB 2.0 OTG dual role device controller which integrates the (OHCI) host controller, device controller, and I²C. The I²C interface that is part of the USB block is intended to control an external OTG transceiver, and is not the same as the I²C peripherals described in [Section 19.1](#).

USB OTG (On-The-Go) is a supplement to the USB 2.0 specification that augments the capability of existing mobile devices and USB peripherals by adding host functionality for connection to USB peripherals. The specification and more information on USB OTG can be found on the USB Implementers Forum web site.

13.4 Features

- Fully compliant with On-The-Go supplement to the *USB 2.0 Specification, Revision 1.0a*.
- Hardware support for Host Negotiation Protocol (HNP).
- Includes a programmable timer required for HNP and SRP.
- Supports any OTG transceiver compliant with the *OTG Transceiver Specification (CEA-2011), Rev. 1.0*.

13.5 Architecture

The architecture of the USB OTG controller is shown below in the block diagram.

The host, device, OTG, and I²C controllers can be programmed through the register interface. The OTG controller enables dynamic switching between host and device roles through the HNP protocol. One port may be connected to an external OTG transceiver to support an OTG connection. The communication between the register interface and an external OTG transceiver is handled through an I²C interface and through the external OTG transceiver interrupt signal.

For USB connections that use the device or host controller only (not OTG), the ports use an embedded USB Analog Transceiver (ATX).

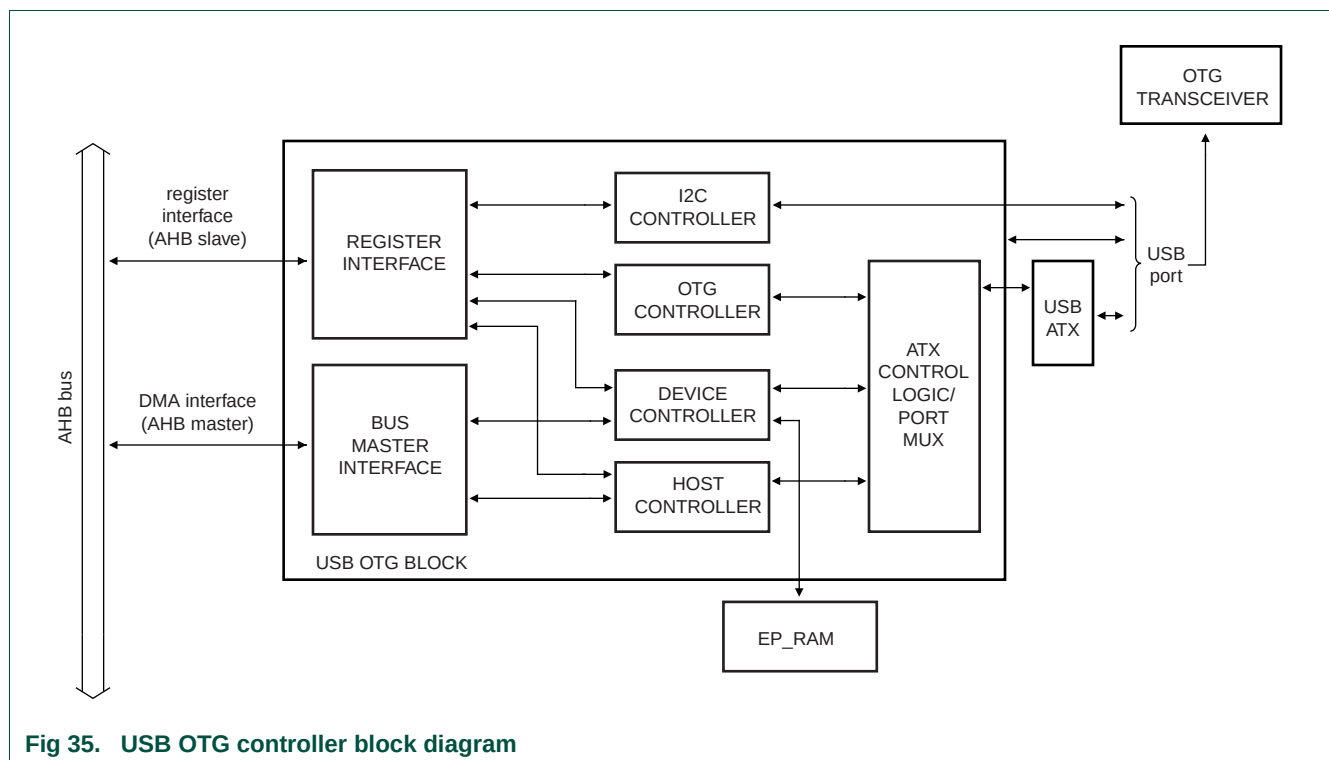


Fig 35. USB OTG controller block diagram

13.6 Modes of operation

The OTG controller is capable of operating in the following modes:

- Host mode (see [Figure 36](#))
- Device mode (see [Figure 37](#))
- OTG mode (see [Figure 38](#))

13.7 Pin configuration

The OTG controller has one USB port.

Table 256. USB OTG port pins

Pin name	Direction	Description	Pin category
USB_D+	I/O	Positive differential data	USB Connector
USB_D-	I/O	Negative differential data	USB Connector
USB_UP_LED	O	GoodLink LED control signal	Control
USB_SCL	I/O	I ² C serial clock	External OTG transceiver
USB_SDA	I/O	I ² C serial data	External OTG transceiver

The following figures show different ways to realize connections to an USB device. The example described here uses an ISP1302 (ST-Ericsson) for the external OTG transceiver and the USB Host power switch LM3526-L (National Semiconductors).

13.7.1 Connecting the USB port to an external OTG transceiver

For OTG functionality an external OTG transceiver must be connected to the LPC176x/5x: Use the internal USB transceiver for USB signalling and use the external OTG transceiver for OTG functionality only (see [Figure 36](#)). This option uses the internal transceiver in VP/VM mode.

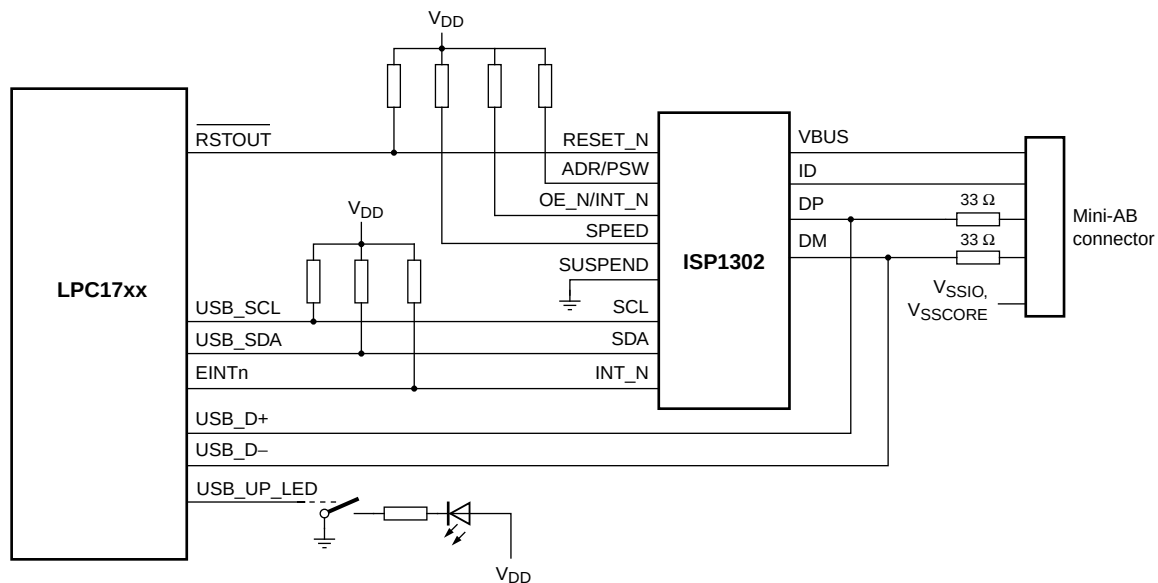


Fig 36. USB OTG port configuration

13.7.2 Connecting USB as a host

The USB port is connected as host using an embedded USB transceiver. There is no OTG functionality on the port.

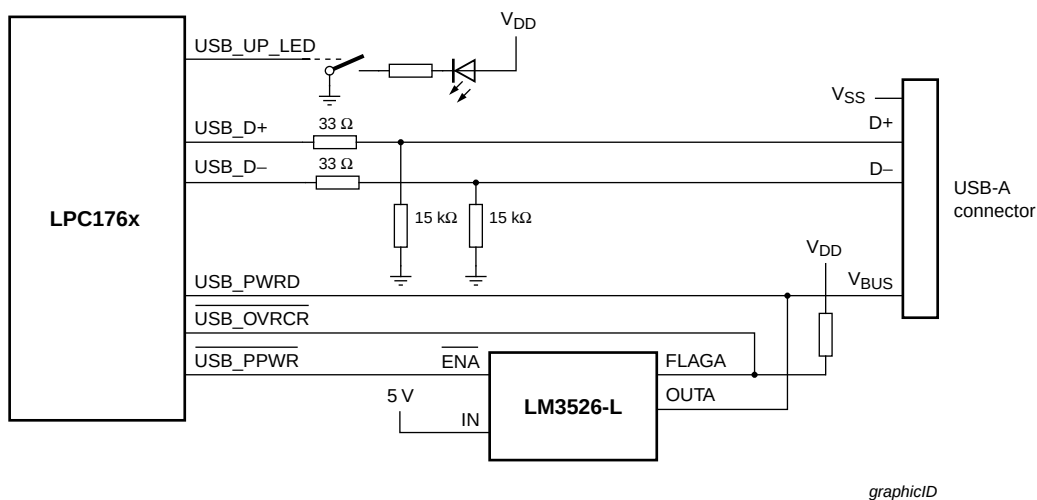


Fig 37. USB host port configuration

13.7.3 Connecting USB as device

The USB port is connected as device. There is no OTG functionality on the USB port.

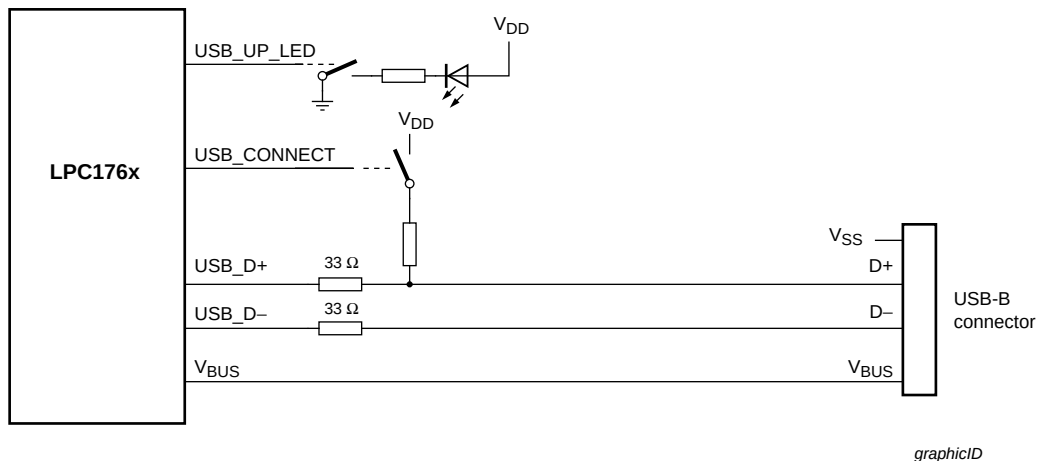


Fig 38. USB device port configuration

13.8 Register description

The OTG and I²C registers are summarized in the following table.

The Device and Host registers are explained in [Table 255](#) and [Table 189](#) in the USB Device Controller and USB Host (OHCI) Controller chapters. All registers are 32 bits wide and aligned to word address boundaries.

Table 257. USB OTG and I²C register address definitions

Name	Description	Access	Reset value	Address
Interrupt register				
USBIntSt	USB Interrupt Status	R/W	0x8000 0100	0x400F C1C0
OTG registers				
OTGIntSt	OTG Interrupt Status	RO	0	0x5000 C100
OTGIntEn	OTG Interrupt Enable	R/W	0	0x5000 C104
OTGIntSet	OTG Interrupt Set	WO	NA	0x5000 C108
OTGIntClr	OTG Interrupt Clear	WO	NA	0x5000 C10C
OTGStCtrl	OTG Status and Control	R/W	0	0x5000 C110
OTGTmr	OTG Timer	R/W	0xFFFF	0x5000 C114
I²C registers				
I2C_RX	I ² C Receive	RO	NA	0x5000 C300
I2C_TX	I ² C Transmit	WO	NA	0x5000 C300
I2C_STS	I ² C Status	RO	0x0A00	0x5000 C304
I2C_CTL	I ² C Control	R/W	0	0x5000 C308
I2C_CLKHI	I ² C Clock High	R/W	0xB9	0x5000 C30C
I2C_CLKLO	I ² C Clock Low	WO	0xB9	0x5000 C310
Clock control registers				
OTGClkCtrl	OTG clock controller	R/W	0	0x5000 CFF4
OTGClkSt	OTG clock status	RO	0	0x5000 CFF8

13.8.1 USB Interrupt Status Register (USBIntSt - 0x5000 C1C0)

The USB OTG controller has seven interrupt lines. This register allows software to determine their status with a single read operation.

The interrupt lines are ORed together to a single channel of the vectored interrupt controller.

Remark: In a device-only configuration bits 6 to 3 of this register are reserved (see [Table 192](#)).

Table 258. USB Interrupt Status register - (USBIntSt - address 0x5000 C1C0) bit description

Bit	Symbol	Description	Reset Value
0	USB_INT_REQ_LP	Low priority interrupt line status. This bit is read-only.	0
1	USB_INT_REQ_HP	High priority interrupt line status. This bit is read-only.	0
2	USB_INT_REQ_DMA	DMA interrupt line status. This bit is read-only.	0

Table 258. USB Interrupt Status register - (USBIntSt - address 0x5000 C1C0) bit description ...continued

Bit	Symbol	Description	Reset Value
3	USB_HOST_INT	USB host interrupt line status. This bit is read-only.	0
4	USB_ATX_INT	External ATX interrupt line status. This bit is read-only.	0
5	USB_OTG_INT	OTG interrupt line status. This bit is read-only.	0
6	USB_I2C_INT	I ² C module interrupt line status. This bit is read-only.	0
7	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
8	USB_NEED_CLK	USB need clock indicator. This bit is read-only.	1
30:9	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
31	EN_USB_INTS	Enable all USB interrupts. When this bit is cleared, the NVIC does not see the ORed output of the USB interrupt lines.	1

13.8.2 OTG Interrupt Status Register (OTGIntSt - 0x5000 C100)

Bits in this register are set by hardware when the interrupt event occurs during the HNP handoff sequence. See [Section 13.9](#) for more information on when these bits are set.

Table 259. OTG Interrupt Status register (OTGIntSt - address 0x5000 C100) bit description

Bit	Symbol	Description	Reset Value
0	TMR	Timer time-out.	0
1	REMOVE_PU	Remove pull-up. This bit is set by hardware to indicate that software needs to disable the D+ pull-up resistor.	0
2	HNP_FAILURE	HNP failed. This bit is set by hardware to indicate that the HNP switching has failed.	0
3	HNP_SUCCESS	HNP succeeded. This bit is set by hardware to indicate that the HNP switching has succeeded.	0
31:4	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.3 OTG Interrupt Enable Register (OTGIntEn - 0x5000 C104)

Writing a one to a bit in this register enables the corresponding bit in OTGIntSt to generate an interrupt on one of the interrupt lines. The interrupt is routed to the USB_OTG_INT interrupt line in the USBIntSt register.

The bit allocation and reset value of OTGIntEn is the same as OTGIntSt.

13.8.4 OTG Interrupt Set Register (OTGIntSet - 0x5000 C20C)

Writing a one to a bit in this register will set the corresponding bit in the OTGIntSt register. Writing a zero has no effect. The bit allocation of OTGIntSet is the same as in OTGIntSt.

13.8.5 OTG Interrupt Clear Register (OTGIntClr - 0x5000 C10C)

Writing a one to a bit in this register will clear the corresponding bit in the OTGIntSt register. Writing a zero has no effect. The bit allocation of OTGIntClr is the same as in OTGIntSt.

13.8.6 OTG Status and Control Register (OTGStCtrl - 0x5000 C110)

The OTGStCtrl register allows enabling hardware tracking during the HNP hand over sequence, controlling the OTG timer, monitoring the timer count, and controlling the functions mapped to port U1 and U2.

Time critical events during the switching sequence are controlled by the OTG timer. The timer can operate in two modes:

1. Monoshot mode: an interrupt is generated at the end of TIMEOUT_CNT (see [Section 13.8.7 “OTG Timer Register \(OTGTmr - 0x5000 C114\)”](#)), the TMR bit is set in OTGIntSt, and the timer will be disabled.
2. Free running mode: an interrupt is generated at the end of TIMEOUT_CNT (see [Section 13.8.7 “OTG Timer Register \(OTGTmr - 0x5000 C114\)”](#)), the TMR bit is set, and the timer value is reloaded into the counter. The timer is not disabled in this mode.

Table 260. OTG Status Control register (OTGStCtrl - address 0x5000 C110) bit description

Bit	Symbol	Description	Reset Value
1:0	PORT_FUNC	Controls port function. Bit 0 is set or cleared by hardware - when B_HNP_TRACK or A_HNP_TRACK is set and HNP succeeds. See Section 13.9 . Bit 1 is reserved.	-
3:2	TMR_SCALE	Timer scale selection. This field determines the duration of each timer count. 00: 10 μ s (100 KHz) 01: 100 μ s (10 KHz) 10: 1000 μ s (1 KHz) 11: Reserved	0x0
4	TMR_MODE	Timer mode selection. 0: monoshot 1: free running	0
5	TMR_EN	Timer enable. When set, TMR_CNT increments. When cleared, TMR_CNT is reset to 0.	0
6	TMR_RST	Timer reset. Writing one to this bit resets TMR_CNT to 0. This provides a single bit control for the software to restart the timer when the timer is enabled.	0
7	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

Table 260. OTG Status Control register (OTGStCtrl - address 0x5000 C110) bit description ...continued

Bit	Symbol	Description	Reset Value
8	B_HNP_TRACK	Enable HNP tracking for B-device (peripheral), see Section 13.9 . Hardware clears this bit when HNP_SUCCESS or HNP_FAILURE is set.	0
9	A_HNP_TRACK	Enable HNP tracking for A-device (host), see Section 13.9 . Hardware clears this bit when HNP_SUCCESS or HNP_FAILURE is set.	0
10	PU_REMOVED	When the B-device changes its role from peripheral to host, software sets this bit when it removes the D+ pull-up, see Section 13.9 . Hardware clears this bit when HNP_SUCCESS or HNP_FAILURE is set.	0
15:11	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
31:16	TMR_CNT	Current timer count value.	0x0

13.8.7 OTG Timer Register (OTGTmr - 0x5000 C114)

Table 261. OTG Timer register (OTGTmr - address 0x5000 C114) bit description

Bit	Symbol	Description	Reset Value
15:0	TIMEOUT_CNT	The TMR interrupt is set when TMR_CNT reaches this value.	0xFFFF
31:16	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.8 OTG Clock Control Register (OTGClkCtrl - 0x5000 CFF4)

This register controls the clocking of the OTG controller. Whenever software wants to access the registers, the corresponding clock control bit needs to be set. The software does not have to repeat this exercise for every register access, provided that the corresponding OTGClkCtrl bits are already set.

Table 262. OTG clock control register (OTG_clock_control - address 0x5000 CFF4) bit description

Bit	Symbol	Value	Description	Reset Value
0	HOST_CLK_EN		Host clock enable	0
		0	Disable the Host clock.	
		1	Enable the Host clock.	
1	DEV_CLK_EN		Device clock enable	0
		0	Disable the Device clock.	
		1	Enable the Device clock.	
2	I2C_CLK_EN		I ² C clock enable	0
		0	Disable the I ² C clock.	
		1	Enable the I ² C clock.	

Table 262. OTG clock control register (OTG_clock_control - address 0x5000 CFF4) bit description ...continued

Bit	Symbol	Value	Description	Reset Value
3	OTG_CLK_EN		OTG clock enable	0
		0	Disable the OTG clock.	
		1	Enable the OTG clock.	
4	AHB_CLK_EN		AHB master clock enable	0
		0	Disable the AHB clock.	
		1	Enable the AHB clock.	
31:5	-	NA	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.9 OTG Clock Status Register (OTGClkSt - 0x5000 CFF8)

This register holds the clock availability status. When enabling a clock via OTGClkCtrl, software should poll the corresponding bit in this register. If it is set, then software can go ahead with the register access. Software does not have to repeat this exercise for every access, provided that the OTGClkCtrl bits are not disturbed.

Table 263. OTG clock status register (OTGClkSt - address 0x5000 CFF8) bit description

Bit	Symbol	Value	Description	Reset Value
0	HOST_CLK_ON		Host clock status.	0
		0	Host clock is not available.	
		1	Host clock is available.	
1	DEV_CLK_ON		Device clock status.	0
		0	Device clock is not available.	
		1	Device clock is available.	
2	I2C_CLK_ON		I ² C clock status.	0
		0	I ² C clock is not available.	
		1	I ² C clock is available.	
3	OTG_CLK_ON		OTG clock status.	0
		0	OTG clock is not available.	
		1	OTG clock is available.	
4	AHB_CLK_ON		AHB master clock status.	0
		0	AHB clock is not available.	
		1	AHB clock is available.	
31:5	-	NA	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.10 I²C Receive Register (I2C_RX - 0x5000 C300)

This register is the top byte of the receive FIFO. The receive FIFO is 4 bytes deep. The Rx FIFO is flushed by a hard reset or by a soft reset (I2C_CTL bit 7). Reading an empty FIFO gives unpredictable data results.

Table 264. I²C Receive register (I2C_RX - address 0x5000 C300) bit description

Bit	Symbol	Description	Reset Value
7:0	RX Data	Receive data.	-

13.8.11 I²C Transmit Register (I2C_TX - 0x5000 C300)

This register is the top byte of the transmit FIFO. The transmit FIFO is 4 bytes deep.

The Tx FIFO is flushed by a hard reset, soft reset (I2C_CTL bit 7) or if an arbitration failure occurs (I2C_STS bit 3). Data writes to a full FIFO are ignored.

I2C_TX must be written for both write and read operations to transfer each byte. Bits [7:0] are ignored for master-receive operations. The master-receiver must write a dummy byte to the TX FIFO for each byte it expects to receive in the RX FIFO. When the STOP bit is set or the START bit is set to cause a RESTART condition on a byte written to the TX FIFO (master-receiver), then the byte read from the slave is not acknowledged. That is, the last byte of a master-receive operation is not acknowledged.

Table 265. I²C Transmit register (I2C_TX - address 0x5000 C300) bit description

Bit	Symbol	Description	Reset Value
7:0	TX Data	Transmit data.	-
8	START	When 1, issue a START condition before transmitting this byte.	-
9	STOP	When 1, issue a STOP condition after transmitting this byte.	-
31:10	-	Reserved. User software should not write ones to reserved bits. The value read from a reserved bit is not defined.	-

13.8.12 I²C Status Register (I2C_STS - 0x5000 C304)

The I2C_STS register provides status information on the TX and RX blocks as well as the current state of the external buses. Individual bits are enabled as interrupts by the I2C_CTL register and routed to the I2C_USB_INT bit in USBIntSt.

Table 266. I²C status register (I2C_STS - address 0x5000 C304) bit description

Bit	Symbol	Value	Description	Reset Value
0	TDI		Transaction Done Interrupt. This flag is set if a transaction completes successfully. It is cleared by writing a one to bit 0 of the status register. It is unaffected by slave transactions.	0
		0	Transaction has not completed.	
		1	Transaction completed.	
1	AFI		Arbitration Failure Interrupt. When transmitting, if the SDA is low when SDAOUT is high, then this I ² C has lost the arbitration to another device on the bus. The Arbitration Failure bit is set when this happens. It is cleared by writing a one to bit 1 of the status register.	0
		0	No arbitration failure on last transmission.	
		1	Arbitration failure occurred on last transmission.	

Table 266. I²C status register (I2C_STS - address 0x5000 C304) bit description ...continued

Bit	Symbol	Value	Description	Reset Value
2	NAI		No Acknowledge Interrupt. After every byte of data is sent, the transmitter expects an acknowledge from the receiver. This bit is set if the acknowledge is not received. It is cleared when a byte is written to the master TX FIFO.	0
		0	Last transmission received an acknowledge.	
		1	Last transmission did not receive an acknowledge.	
3	DRMI		Master Data Request Interrupt. Once a transmission is started, the transmitter must have data to transmit as long as it isn't followed by a stop condition or it will hold SCL low until more data is available. The Master Data Request bit is set when the master transmitter is data-starved. If the master TX FIFO is empty and the last byte did not have a STOP condition flag, then SCL is held low until the CPU writes another byte to transmit. This bit is cleared when a byte is written to the master TX FIFO.	0
		0	Master transmitter does not need data.	
		1	Master transmitter needs data.	
4	DRSI		Slave Data Request Interrupt. Once a transmission is started, the transmitter must have data to transmit as long as it isn't followed by a STOP condition or it will hold SCL low until more data is available. The Slave Data Request bit is set when the slave transmitter is data-starved. If the slave TX FIFO is empty and the last byte transmitted was acknowledged, then SCL is held low until the CPU writes another byte to transmit. This bit is cleared when a byte is written to the slave Tx FIFO.	0
		0	Slave transmitter does not need data.	
		1	Slave transmitter needs data.	
5	Active		Indicates whether the bus is busy. This bit is set when a START condition has been seen. It is cleared when a STOP condition is seen..	0
6	SCL		The current value of the SCL signal.	-
7	SDA		The current value of the SDA signal.	-
8	RFF		Receive FIFO Full (RFF). This bit is set when the RX FIFO is full and cannot accept any more data. It is cleared when the RX FIFO is not full. If a byte arrives when the Receive FIFO is full, the SCL is held low until the CPU reads the RX FIFO and makes room for it.	0
		0	RX FIFO is not full	
		1	RX FIFO is full	
9	RFE		Receive FIFO Empty. RFE is set when the RX FIFO is empty and is cleared when the RX FIFO contains valid data.	1
		0	RX FIFO contains data.	
		1	RX FIFO is empty	
10	TFF		Transmit FIFO Full. TFF is set when the TX FIFO is full and is cleared when the TX FIFO is not full.	0
		0	TX FIFO is not full.	
		1	TX FIFO is full	

Table 266. I²C status register (I2C_STS - address 0x5000 C304) bit description ...continued

Bit	Symbol	Value	Description	Reset Value
11	TFE		Transmit FIFO Empty. TFE is set when the TX FIFO is empty and is cleared when the TX FIFO contains valid data.	1
		0	TX FIFO contains valid data.	
		1	TX FIFO is empty	
31:12	-	NA	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.13 I²C Control Register (I2C_CTL - 0x5000 C308)

The I2C_CTL register is used to enable interrupts and reset the I²C state machine. Enabled interrupts cause the USB_I2C_INT interrupt output line to be asserted when set.

Table 267. I²C Control register (I2C_CTL - address 0x5000 C308) bit description

Bit	Symbol	Value	Description	Reset Value
0	TDIE		Transmit Done Interrupt Enable. This enables the TDI interrupt signalling that this I ² C issued a STOP condition.	0
		0	Disable the TDI interrupt.	
		1	Enable the TDI interrupt.	
1	AFIE		Transmitter Arbitration Failure Interrupt Enable. This enables the AFI interrupt which is asserted during transmission when trying to set SDA high, but the bus is driven low by another device.	0
		0	Disable the AFI.	
		1	Enable the AFI.	
2	NAIE		Transmitter No Acknowledge Interrupt Enable. This enables the NAI interrupt signalling that transmitted byte was not acknowledged.	0
		0	Disable the NAI.	
		1	Enable the NAI.	
3	DRMIE		Master Transmitter Data Request Interrupt Enable. This enables the DRMI interrupt which signals that the master transmitter has run out of data, has not issued a STOP, and is holding the SCL line low.	0
		0	Disable the DRMI interrupt.	
		1	Enable the DRMI interrupt.	
4	DRSIE		Slave Transmitter Data Request Interrupt Enable. This enables the DRSI interrupt which signals that the slave transmitter has run out of data and the last byte was acknowledged, so the SCL line is being held low.	0
		0	Disable the DRSI interrupt.	
		1	Enable the DRSI interrupt.	
5	REFIE		Receive FIFO Full Interrupt Enable. This enables the Receive FIFO Full interrupt to indicate that the receive FIFO cannot accept any more data.	0
		0	Disable the RFFI.	
		1	Enable the RFFI.	

Table 267. I²C Control register (I2C_CTL - address 0x5000 C308) bit description ...continued

Bit	Symbol	Value	Description	Reset Value
6	RFDAIE		Receive Data Available Interrupt Enable. This enables the DAI interrupt to indicate that data is available in the receive FIFO (i.e. not empty).	0
		0	Disable the DAI.	
		1	Enable the DAI.	
7	TFFIE		Transmit FIFO Not Full Interrupt Enable. This enables the Transmit FIFO Not Full interrupt to indicate that the more data can be written to the transmit FIFO. Note that this is not full. It is intended help the CPU to write to the I ² C block only when there is room in the FIFO and do this without polling the status register.	0
		0	Disable the TFFI.	
		1	Enable the TFFI.	
8	SRST		Soft reset. This is only needed in unusual circumstances. If a device issues a start condition without issuing a stop condition. A system timer may be used to reset the I ² C if the bus remains busy longer than the time-out period. On a soft reset, the Tx and Rx FIFOs are flushed, I2C_STS register is cleared, and all internal state machines are reset to appear idle. The I2C_CLKHI, I2C_CLKLO and I2C_CTL (except Soft Reset Bit) are NOT modified by a soft reset.	0
		0	See the text.	
		1	Reset the I ² C to idle state. Self clearing.	
31:9	-	NA	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

13.8.14 I²C Clock High Register (I2C_CLKHI - 0x5000 C30C)

The CLK register holds a terminal count for counting 48 MHz clock cycles to create the high period of the slower I²C serial clock, SCL.

Table 268. I²C_CLKHI register (I2C_CLKHI - address 0x5000 C30C) bit description

Bit	Symbol	Description	Reset Value
7:0	CDHI	Clock divisor high. This value is the number of 48 MHz clocks the serial clock (SCL) will be high.	0xB9

13.8.15 I²C Clock Low Register (I2C_CLKLO - 0x5000 C310)

The CLK register holds a terminal count for counting 48 MHz clock cycles to create the low period of the slower I²C serial clock, SCL.

Table 269. I²C_CLKLO register (I2C_CLKLO - address 0x5000 C310) bit description

Bit	Symbol	Description	Reset Value
7:0	CDLO	Clock divisor low. This value is the number of 48 MHz clocks the serial clock (SCL) will be low.	0xB9

13.8.16 Interrupt handling

The interrupts set in the OTGIntSt register are set and cleared during HNP switching. All OTG related interrupts, if enabled, are routed to the USB_OTG_INT bit in the USBIntSt register.

I²C related interrupts are set in the I2C_STS register and routed, if enabled by I2C_CTL, to the USB_I2C_INT bit.

For more details on the interrupts created by device controller, see the USB device chapter. For interrupts created by the host controllers, see the OHCI specification.

The EN_USB_INTS bit in the USBIntSt register enables the routing of any of the USB related interrupts to the NVIC controller (see [Figure 39](#)).

Remark: During the HNP switching between host and device with the OTG stack active, an action may raise several levels of interrupts. It is advised to let the OTG stack initiate any actions based on interrupts and ignore device and host level interrupts. This means that during HNP switching, the OTG stack provides the communication to the host and device controllers.

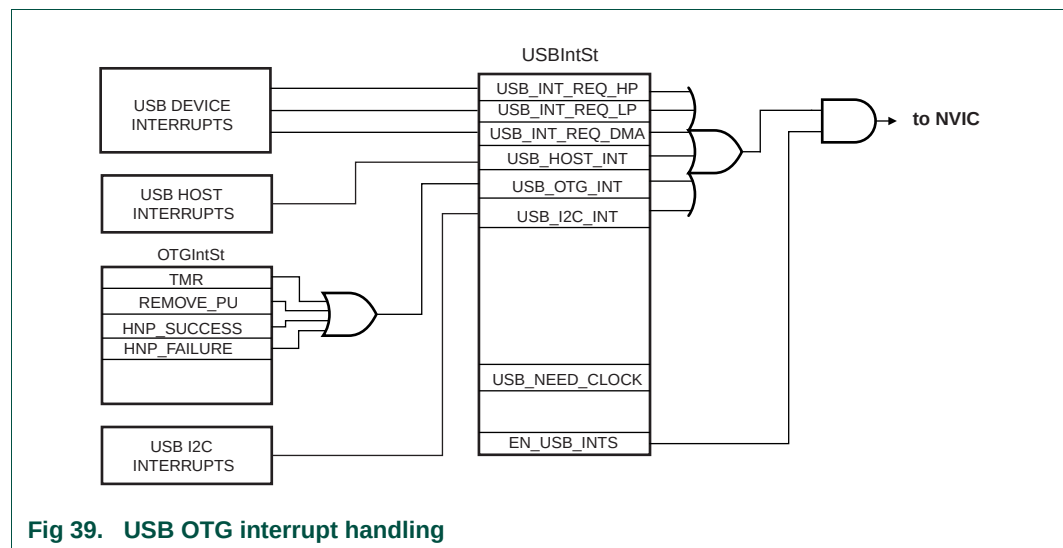


Fig 39. USB OTG interrupt handling

13.9 HNP support

This section describes the hardware support for the Host Negotiation Protocol (HNP) provided by the OTG controller.

When two dual-role OTG devices are connected to each other, the plug inserted into the mini-AB receptacle determines the default role of each device. The device with the mini-A plug inserted becomes the default Host (A-device), and the device with the mini-B plug inserted becomes the default Peripheral (B-device).

Once connected, the default Host (A-device) and the default Peripheral (B-device) can switch Host and Peripheral roles using HNP.

The context of the OTG controller operation is shown in [Figure 40](#). Each controller (Host, Device, or OTG) communicates with its software stack through a set of status and control registers and interrupts. In addition, the OTG software stack communicates with the external OTG transceiver through the I²C interface and the external transceiver interrupt signal.

The OTG software stack is responsible for implementing the HNP state machines as described in the On-The-Go Supplement to the USB 2.0 Specification.

The OTG controller hardware provides support for some of the state transitions in the HNP state machines as described in the following subsections.

The USB state machines, the HNP switching, and the communications between the USB controllers are described in more detail in the following documentation:

- USB OHCI specification
- USB OTG supplement, version 1.2
- USB 2.0 specification
- ISP1302 data sheet and user manual

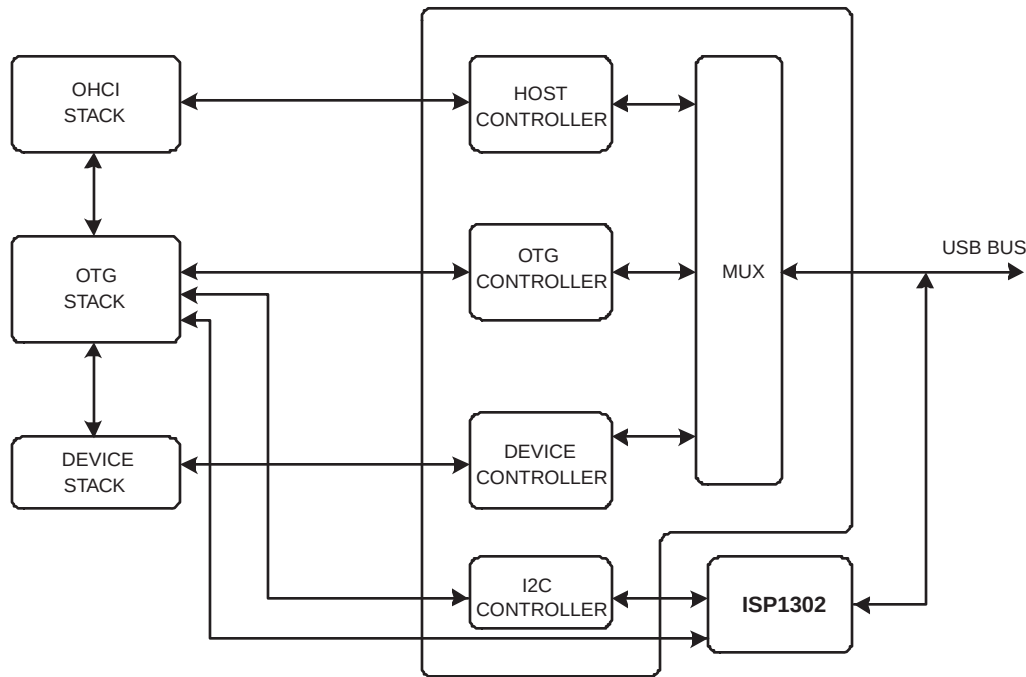


Fig 40. USB OTG controller with software stack

13.9.1 B-device: peripheral to host switching

In this case, the default role of the OTG controller is peripheral (B-device), and it switches roles from Peripheral to Host.

The On-The-Go Supplement defines the behavior of a dual-role B-device during HNP using a state machine diagram. The OTG software stack is responsible for implementing all of the states in the Dual-Role B-Device State Diagram.

The OTG controller hardware provides support for the state transitions between the states `b_peripheral`, `b_wait_acon`, and `b_host` in the Dual-Role B-Device state diagram. Setting `B_HNP_TRACK` in the `OTGStCtrl` register enables hardware support for the B-device switching from peripheral to host. The hardware actions after setting this bit are shown in [Figure 41](#).

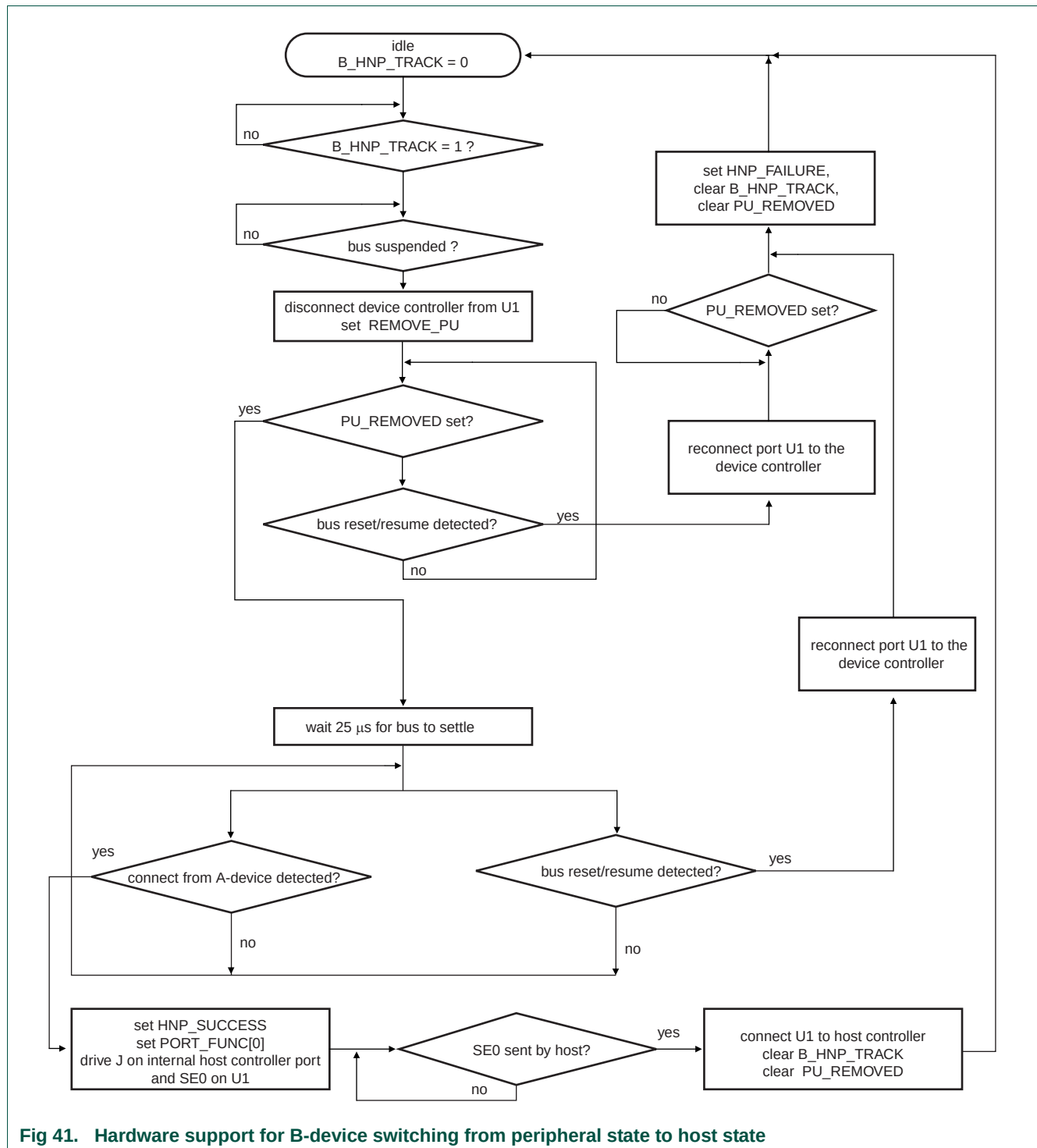


Fig 41. Hardware support for B-device switching from peripheral state to host state

Figure 42 shows the actions that the OTG software stack should take in response to the hardware actions setting REMOVE_PU, HNP_SUCCESS, AND HNP_FAILURE. The relationship of the software actions to the Dual-Role B-Device states is also shown. B-device states are in bold font with a circle around them.

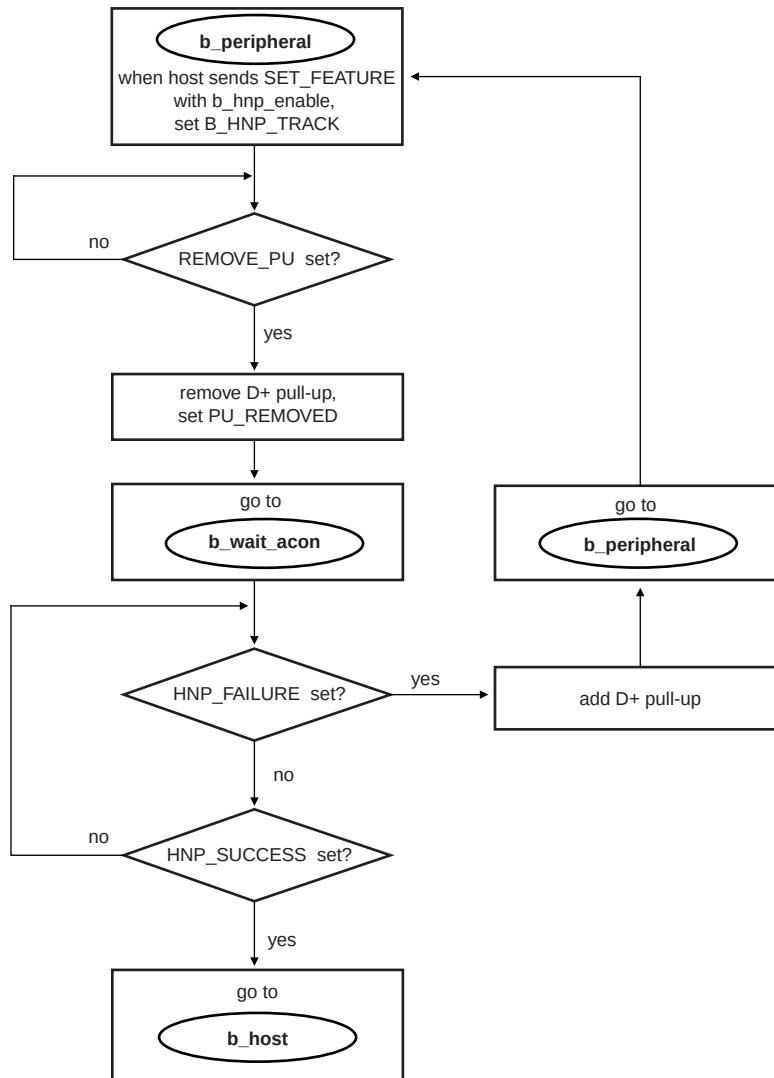


Fig 42. State transitions implemented in software during B-device switching from peripheral to host

Note that only the subset of B-device HNP states and state transitions supported by hardware are shown. Software is responsible for implementing all of the HNP states.

[Figure 42](#) may appear to imply that the interrupt bits such as REMOVE_PU should be polled, but this is not necessary if the corresponding interrupt is enabled.

Following are code examples that show how the actions in [Figure 42](#) are accomplished. The examples assume that ISP1302 is being used as the external OTG transceiver.

Remove D+ pull-up

```

/* Remove D+ pull-up through ISP1302 */
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0
OTG_I2C_TX = 0x007; // Send OTG Control (Clear) register address
OTG_I2C_TX = 0x201; // Clear DP_PULLUP bit, send STOP condition

```

```
/* Wait for TDI to be set */  
while (!(OTG_I2C_STS & TDI));
```

```
/* Clear TDI */  
OTG_I2C_STS = TDI;
```

Add D+ pull-up

```
/* Add D+ pull-up through ISP1302 */  
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0  
OTG_I2C_TX = 0x006; // Send OTG Control (Set) register address  
OTG_I2C_TX = 0x201; // Set DP_PULLUP bit, send STOP condition
```

```
/* Wait for TDI to be set */  
while (!(OTG_I2C_STS & TDI));
```

```
/* Clear TDI */  
OTG_I2C_STS = TDI;
```

13.9.2 A-device: host to peripheral HNP switching

In this case, the role of the OTG controller is host (A-device), and the A-device switches roles from host to peripheral.

The On-The-Go Supplement defines the behavior of a dual-role A-device during HNP using a state machine diagram. The OTG software stack is responsible for implementing all of the states in the Dual-Role A-Device State Diagram.

The OTG controller hardware provides support for the state transitions between a_host, a_suspend, a_wait_vfall, and a_peripheral in the Dual-Role A-Device state diagram. Setting A_HNP_TRACK in the OTGStCtrl register enables hardware support for switching the A-device from the host state to the device state. The hardware actions after setting this bit are shown in [Figure 43](#).

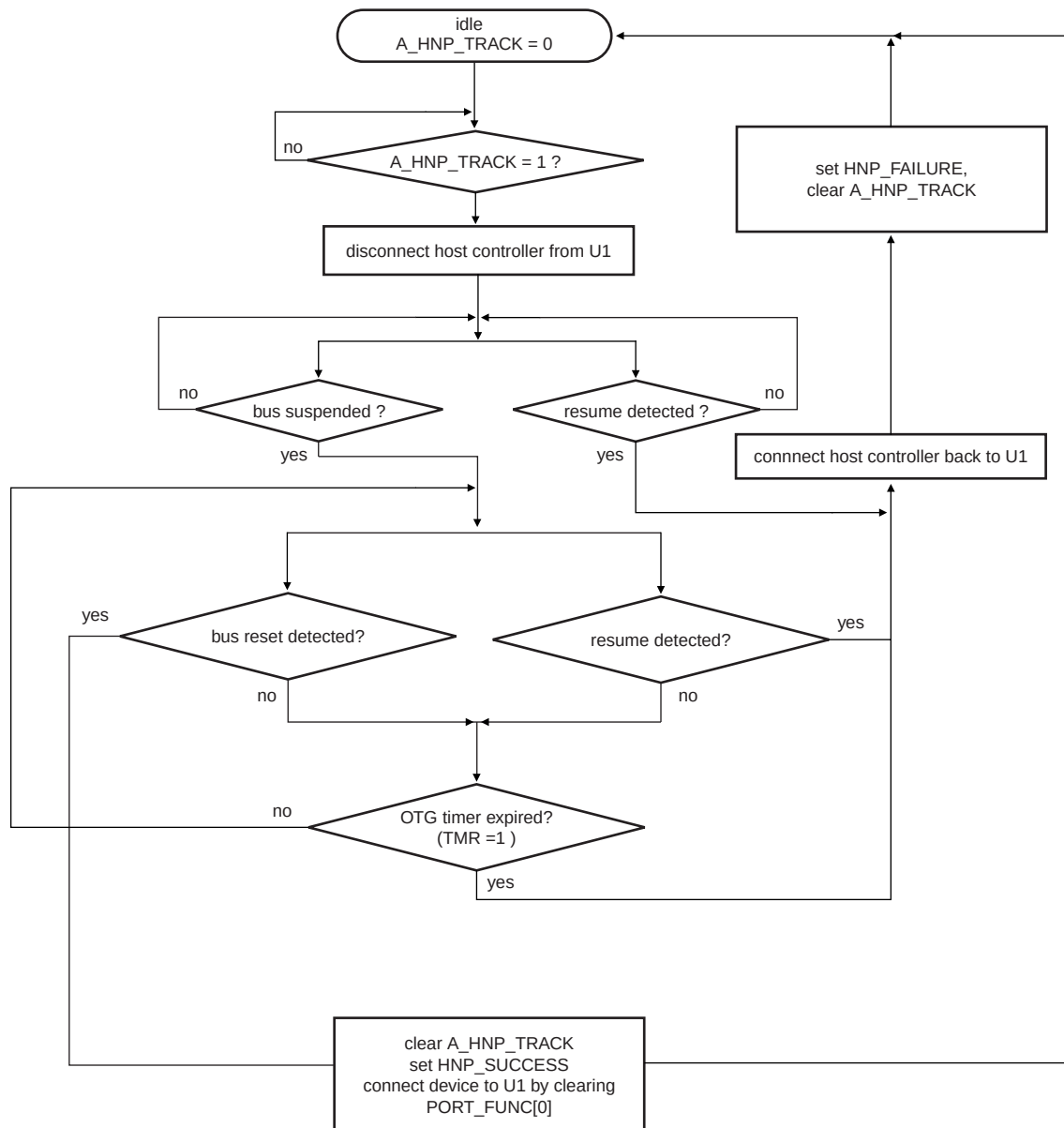


Fig 43. Hardware support for A-device switching from host state to peripheral state

[Figure 44](#) shows the actions that the OTG software stack should take in response to the hardware actions setting TMR, HNP_SUCCESS, and HNP_FAILURE. The relationship of the software actions to the Dual-Role A-Device states is also shown. A-device states are shown in bold font with a circle around them.

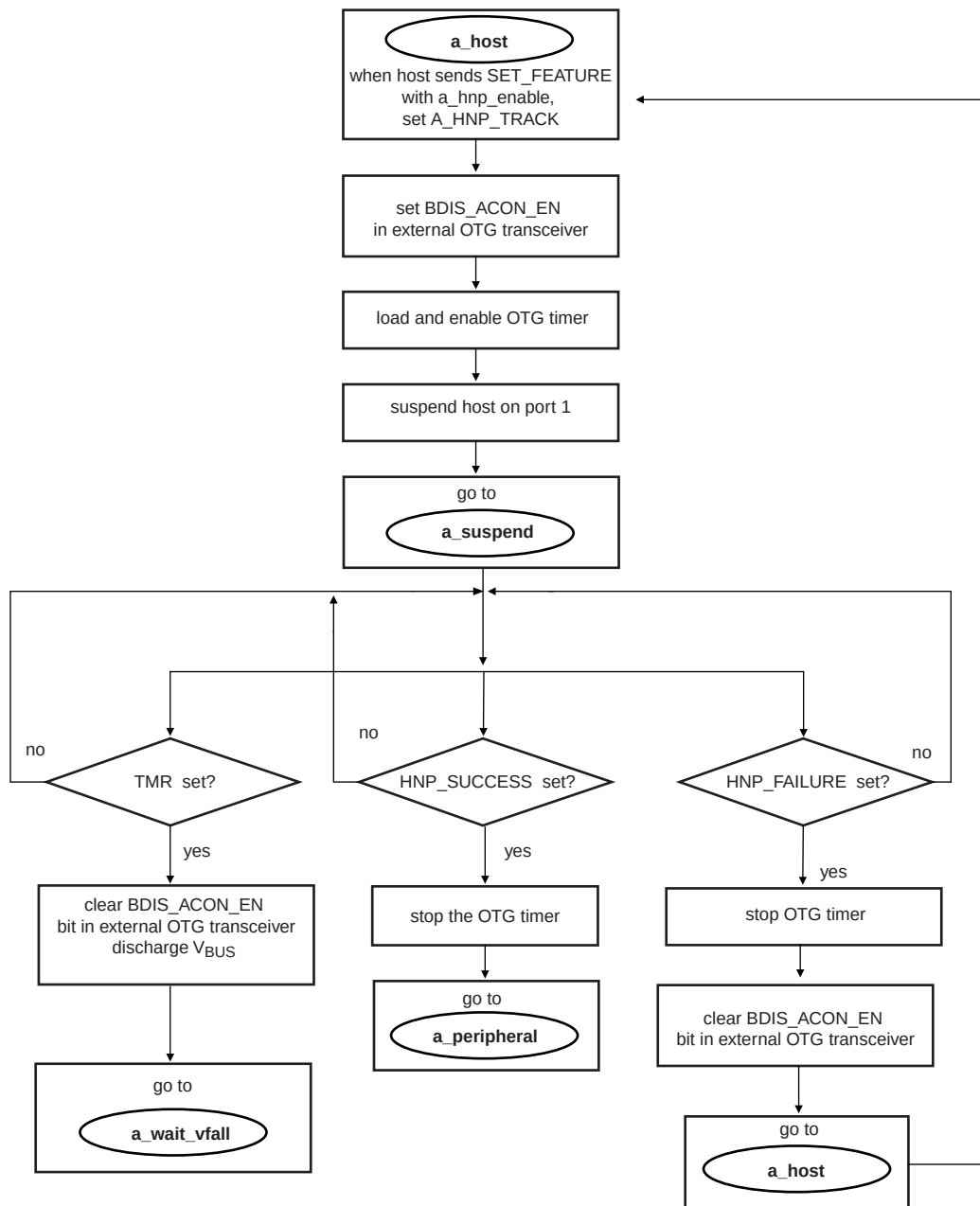


Fig 44. State transitions implemented in software during A-device switching from host to peripheral

Note that only the subset of A-device HNP states and state transitions supported by hardware are shown. Software is responsible for implementing all of the HNP states.

[Figure 44](#) may appear to imply that the interrupt bits such as TMR should be polled, but this is not necessary if the corresponding interrupt is enabled.

Following are code examples that show how the actions in [Figure 44](#) are accomplished. The examples assume that ISP1302 is being used as the external OTG transceiver.

Set BDIS_ACON_EN in external OTG transceiver

```
/* Set BDIS_ACON_EN in ISP1302 */
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0
OTG_I2C_TX = 0x004; // Send Mode Control 1 (Set) register address
OTG_I2C_TX = 0x210; // Set BDIS_ACON_EN bit, send STOP condition

/* Wait for TDI to be set */
while (!(OTG_I2C_STS & TDI));

/* Clear TDI */
OTG_I2C_STS = TDI;
```

Clear BDIS_ACON_EN in external OTG transceiver

```
/* Set BDIS_ACON_EN in ISP1302 */
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0
OTG_I2C_TX = 0x005; // Send Mode Control 1 (Clear) register address
OTG_I2C_TX = 0x210; // Clear BDIS_ACON_EN bit, send STOP condition

/* Wait for TDI to be set */
while (!(OTG_I2C_STS & TDI));

/* Clear TDI */
OTG_I2C_STS = TDI;
```

Discharge V_{BUS}

```
/* Clear the VBUS_DRV bit in ISP1302 */
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0
OTG_I2C_TX = 0x007; // Send OTG Control (Clear) register address
OTG_I2C_TX = 0x220; // Clear VBUS_DRV bit, send STOP condition

/* Wait for TDI to be set */
while (!(OTG_I2C_STS & TDI));

/* Clear TDI */
OTG_I2C_STS = TDI;

/* Set the VBUS_DISCHRG bit in ISP1302 */
OTG_I2C_TX = 0x15A; // Send ISP1302 address, R/W=0
OTG_I2C_TX = 0x006; // Send OTG Control (Set) register address
OTG_I2C_TX = 0x240; // Set VBUS_DISCHRG bit, send STOP condition

/* Wait for TDI to be set */
while (!(OTG_I2C_STS & TDI));

/* Clear TDI */
OTG_I2C_STS = TDI;
```

Load and enable OTG timer

```

/* The following assumes that the OTG timer has previously been */
/* configured for a time scale of 1 ms (TMR_SCALE = "10")      */
/* and monoshot mode (TMR_MODE = 0)                            */

/* Load the timeout value to implement the a_aidl_bdis_tmr timer */
/* the minimum value is 200 ms                                   */
OTG_TIMER = 200;

/* Enable the timer */
OTG_STAT_CTRL |= TMR_EN;

```

Stop OTG timer

```

/* Disable the timer - causes TMR_CNT to be reset to 0 */
OTG_STAT_CTRL &= ~TMR_EN;

/* Clear TMR interrupt */
OTG_INT_CLR = TMR;

```

Suspend host on port 1

```

/* Write to PortSuspendStatus bit to suspend host port 1 - */
/* this example demonstrates the low-level action software needs to take. */
/* The host stack code where this is done will be somewhat more involved. */
HC_RH_PORT_STAT1 = PSS;

```

13.10 Clocking and power management

The OTG controller clocking is shown in [Figure 45](#).

A clock switch controls each clock with the exception of ahb_slave_clk. When the enable of the clock switch is asserted, its clock output is turned on and its CLK_ON output is asserted. The CLK_ON signals are observable in the OTGClkSt register.

To conserve power, the clocks to the Device, Host, OTG, and I²C controllers can be disabled when not in use by clearing the respective CLK_EN bit in the OTGClkCtrl register. When the entire USB block is not in use, all of its clocks can be disabled by clearing the PCUSB bit in the PCONP register.

When software wishes to access registers in one of the controllers, it should first ensure that the respective controller's 48 MHz clock is enabled by setting its CLK_EN bit in the OTGClkCtrl register and then poll the corresponding CLK_ON bit in OTGClkSt until set. Once set, the controller's clock will remain enabled until CLK_EN is cleared by software. Accessing the register of a controller when its 48 MHz clock is not enabled will result in a data abort exception.

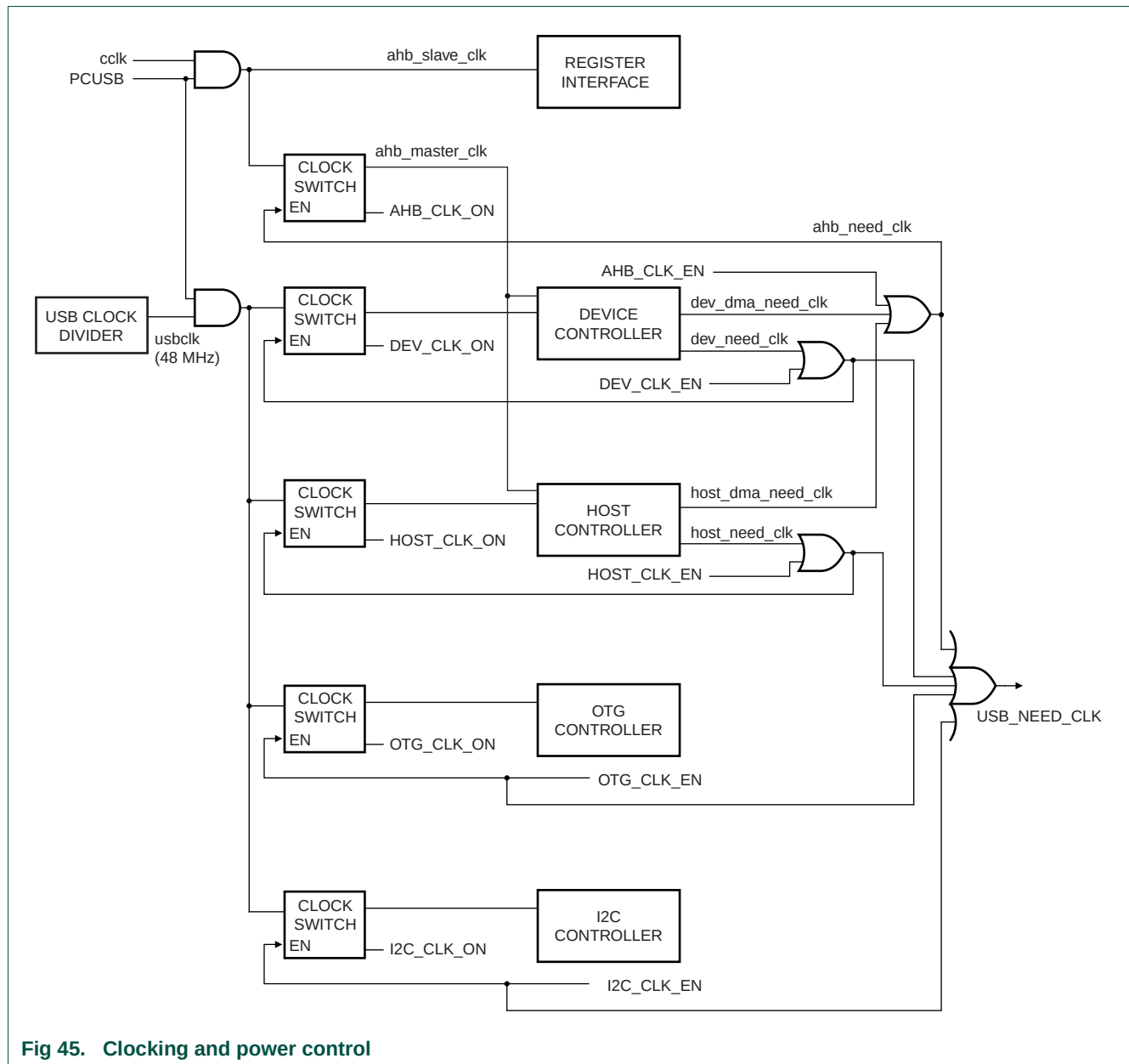


Fig 45. Clocking and power control

13.10.1 Device clock request signals

The Device controller has two clock request signals, `dev_need_clk` and `dev_dma_need_clk`. When asserted, these signals turn on the device's 48 MHz clock and `ahb_master_clk` respectively.

The `dev_need_clk` signal is asserted while the device is not in the suspend state, or if the device is in the suspend state and activity is detected on the USB bus. The `dev_need_clk` signal is de-asserted if a disconnect is detected (CON bit is cleared in the SIE Get Device Status register – [Section 11.10.6](#)). This signal allows `DEV_CLK_EN` to be cleared during normal operation when software does not need to access the Device controller registers – the Device will continue to function normally and automatically shut off its clock when it is suspended or disconnected.

The `dev_dma_need_clk` signal is asserted on any Device controller DMA access to memory. Once asserted, it remains active for 2 ms (2 frames), to help assure that DMA throughput is not affected by any latency associated with re-enabling `ahb_master_clk`. 2 ms after the last DMA access, `dev_dma_need_clk` is de-asserted to help conserve power. This signal allows `AHB_CLK_EN` to be cleared during normal operation.

13.10.1.1 Host clock request signals

The Host controller has two clock request signals, `host_need_clk` and `host_dma_need_clk`. When asserted, these signals turn on the host's 48 MHz clock and `ahb_master_clk` respectively.

The `host_need_clk` signal is asserted while the Host controller functional state is not `UsbSuspend`, or if the functional state is `UsbSuspend` and resume signaling or a disconnect is detected on the USB bus. This signal allows `HOST_CLK_EN` to be cleared during normal operation when software does not need to access the Host controller registers – the Host will continue to function normally and automatically shut off its clock when it goes into the `UsbSuspend` state.

The `host_dma_need_clk` signal is asserted on any Host controller DMA access to memory. Once asserted, it remains active for 2 ms (2 frames), to help assure that DMA throughput is not affected by any latency associated with re-enabling `ahb_master_clk`. 2 ms after the last DMA access, `host_dma_need_clk` is de-asserted to help conserve power. This signal allows `AHB_CLK_EN` to be cleared during normal operation.

13.10.2 Power-down mode support

The LPC176x/5x can be configured to wake up from Power-down mode on any USB bus activity. When the chip is in Power-down mode and the USB interrupt is enabled, the assertion of `USB_NEED_CLK` causes the chip to wake up from Power-down mode.

Before Power-down mode can be entered when the USB activity interrupt is enabled, `USB_NEED_CLK` must be de-asserted. This is accomplished by clearing all of the `CLK_EN` bits in `OTGClkCtrl` and putting the Host controller into the `UsbSuspend` functional state. If it is necessary to wait for either of the `dma_need_clk` signals or the `dev_need_clk` to be de-asserted, the status of `USB_NEED_CLK` can be polled in the `USBIntSt` register to determine when they have all been de-asserted.

13.11 USB OTG controller initialization

The LPC176x/5x OTG device controller initialization includes the following steps:

1. Enable the device controller by setting the `PCUSB` bit of `PCONP`.
2. Configure and enable the USB PLL (`PLL1`) or Main PLL (`PLL0`) to provide 48 MHz for `usbclk` and the desired frequency for `cclk`. For the procedure for determining the PLL setting and configuration, see [Section 4.5.11 "Procedure for determining PLL0 settings"](#) or [Section 4.6.9 "Procedure for determining PLL1 settings"](#).
3. Enable the desired controller clocks by setting their respective `CLK_EN` bits in the `USBClkCtrl` register. Poll the corresponding `CLK_ON` bits in the `USBClkSt` register until they are set.
4. Enable the desired USB pin functions by writing to the corresponding `PINSEL` registers.

5. Follow the appropriate steps in [Section 11.13 “USB device controller initialization”](#) to initialize the device controller.
6. Follow the guidelines given in the OpenHCI specification for initializing the host controller.